ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 - AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

PROJE RAPORU

Hasan Kalın

20291267

GitHub: @hkln4

İÇİNDEKİLER

1. PROJE 1: VERİ TEMİZLEME VE ETL SÜREÇLERİ 1

1.1.Giriş 1

1.2.Extract: Staging Tablosu ve Hedef Tablo Oluşturma 1

1.3.Transform + Load 2

1.4.Veri Kalitesi Raporu 2

1.5.Sonuç 3

2. PROJE 2: PERFORMANS OPTİMİZASYONU VE İZLEME 4

2.1.Giriş 4

2.2.Index Öncesi Performans Ölçümü 4

2.3.Index Oluşturma 4

2.4.Index Sonrası Performans Ölçümü 5

2.5.Gereksiz Index Tespiti ve Kaldırma..... 5

2.6.Erişim Yönetimi ve Roller 6

2.7.Sonuç 6

1.PROJE 1: VERİ TEMİZLEME VE ETL SÜREÇLERİ

1.1. Giriş

Bu proje kapsamında messy_HR_data.csv (GitHub'a yüklendi) üzerinde PostgreSQL ve DBeaver kullanılarak tam bir ETL süreci uygulandı. Veriler; eksik değerler, metin olarak girilmiş sayılar ve tutarsız tarih formatları gibi çok sayıda kalite sorunu içermekteydi.

1.2.Extract: Staging Tablosu Oluşturma ve Hedef Tablo Oluşturma

```sql
DROP TABLE staging_hr;

CREATE TABLE staging_hr (
    "Name" TEXT,
    "Age" TEXT,
    "Salary" TEXT,
    "Gender" TEXT,
    "Department" TEXT,
    "Position" TEXT,
    "Joining Date" TEXT,
    "Performance Score" TEXT,
    "Email" TEXT,
    "Phone Number" TEXT
);

SELECT * FROM staging_hr LIMIT 10;

-- total kayıt
SELECT COUNT(*) FROM staging_hr;

-- Age'deki kirli veriler
SELECT DISTINCT "Age" FROM staging_hr;

-- Salary'daki kirli veriler
SELECT DISTINCT "Salary" FROM staging_hr;

-- Joining Date formatları
SELECT DISTINCT "Joining Date" FROM staging_hr;

-- Boş email'ler
SELECT COUNT(*) FROM staging_hr WHERE "Email" IS NULL OR TRIM("Email") = '' OR "Email" = 'nan';
```

Ham veriyi olduğu gibi saklamak için tüm sütunlar TEXT tipinde tanımlandı. Veri setindeki; eksik verileri, farklı formatları anlamak için keşifsel veri analizi yapıldı.

Toplam kayıt	Age'deki veriler	Salary'deki veriler	Joining Date formatları	Boş Emailler
1000	40	55000	03-25-2019	390
	35	Nan	01/15/2020	
	Nan	70000	2020/02/20	
	thirty	SIXTY THOUSAND	2019.12.01	
	50	50000	April 5, 2018	
	25	65000		

Görüldüğü üzere Age, Salary ve Joining Date sütunlarında farklı formatta veriler bulunuyor. Transform adımında gerekli işlemlerle bu veriler standartlaştırılacaktır.

```sql
CREATE TABLE clean_hr (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    age INTEGER,
    salary NUMERIC(10,2),
    gender VARCHAR(20),
    department VARCHAR(50),
    position VARCHAR(50),
    joining_date DATE,
    performance_score VARCHAR(5),
    email TEXT,
    phone_number VARCHAR(20),
    loaded_at TIMESTAMP DEFAULT NOW()
);
```

Load adımında temizlenmiş veriyi yükleyeceğimiz clean_hr tablosu uygun veri tipleriyle oluşturuldu.

1.3.Transform + Load

```sql
INSERT INTO clean_hr (
    name, age, salary, gender, department, position,
    joining_date, performance_score, email, phone_number
)
SELECT
    -- Name: baştaki/sondaki boşlukları temizle
    TRIM("Name"),

    -- Age: 'nan' ve 'thirty' gibi metin değerleri NULL yap, geçerli sayıları integer'a çevir
    CASE
        WHEN TRIM("Age") ~ '^\d+$' THEN TRIM("Age")::INTEGER
        ELSE NULL
    END,

    -- Salary: 'NAN' → NULL, 'SIXTY THOUSAND' → 60000
    CASE
        WHEN TRIM("Salary") = 'NAN' THEN NULL
        WHEN TRIM("Salary") = 'SIXTY THOUSAND' THEN 60000
        WHEN TRIM("Salary") ~ '^\d+$' THEN TRIM("Salary")::NUMERIC
        ELSE NULL
    END,

    -- Gender: olduğu gibi
    TRIM("Gender"),
    -- Department
    TRIM("Department"),
    -- Position
    TRIM("Position"),

    -- Joining Date: 5 farklı formatı standart DATE'e çevir
    CASE
        WHEN "Joining Date" ~ '^\d{4}/\d{2}/\d{2}$'
            THEN TO_DATE("Joining Date", 'YYYY/MM/DD')
        WHEN "Joining Date" ~ '^\d{2}/\d{2}/\d{4}$'
            THEN TO_DATE("Joining Date", 'MM/DD/YYYY')
        WHEN "Joining Date" ~ '^\d{4}\.\d{2}\.\d{2}$'
            THEN TO_DATE("Joining Date", 'YYYY.MM.DD')
        WHEN "Joining Date" ~ '^\d{2}-\d{2}-\d{4}$'
            THEN TO_DATE("Joining Date", 'MM-DD-YYYY')
        WHEN "Joining Date" ~ '^[A-Za-z]+ \d+, \d{4}$'
            THEN TO_DATE("Joining Date", 'Month DD, YYYY')
        ELSE NULL
    END,
    -- Performance Score
    TRIM("Performance Score"),
```

Name sütununa TRIM("Name") uygulayarak baştaki ve sondaki boşlukları temizlendi.

Age sütununda TRIM("Age") ~ '^\d+$' ile rakamlardan oluşan değerler kontrol edildi. Geçerliyse INT'e çevirildi.Ayrıca thirty manuel olarak 30'a çevrildi. Geriye kalanlar NULL yapıldı.

Salary sütunuda, SIXTY THO.. 60000 sayısına çevrildi, sadece rakamlardan oluşuyorsa NUMERIC tipine dönüştürüldü. Hiçbiri değilse NULL.

Joining Date sütununda, Regex ile her satırın hangi formatta olduğu tespit edilerek, TO_DATE() ile standart bir formata dönüşüm yapıldı.

Email / Phone sütunuda, TRIM() sonrası geçerli değerler TRIM'lendi. Geriye kalanlar ise NULL yapıldı.

Gender / Department / Position / Performance Score sütunlarına Sadece TRIM() uygulandı.

Ayrıca dönüştürülen tüm veriler temiz clean_hr tablosuna yüklendi.

1.4.Veri Kalitesi Raporu

```sql
-- 1. Genel kayıt sayısı
SELECT COUNT(*) AS toplam_kayit FROM clean_hr;

-- 2. NULL değer sayıları (her sütun için)
SELECT
    COUNT(*) - COUNT(name) AS bos_name,
    COUNT(*) - COUNT(age) AS bos_age,
    COUNT(*) - COUNT(salary) AS bos_salary,
    COUNT(*) - COUNT(email) AS bos_email,
    COUNT(*) - COUNT(phone_number) AS bos_phone,
    COUNT(*) - COUNT(joining_date) AS bos_joining_date
FROM clean_hr;

-- 3. Yaş dağılımı
SELECT MIN(age), MAX(age), ROUND(AVG(age),1) AS ort_yas
FROM clean_hr;

-- 4. Maaş dağılımı
SELECT MIN(salary), MAX(salary), ROUND(AVG(salary),2) AS ort_maas
FROM clean_hr WHERE salary IS NOT NULL;

-- 5. Departmana göre çalışan sayısı
SELECT department, COUNT(*) AS calisan_sayisi
FROM clean_hr
GROUP BY department
ORDER BY calisan_sayisi DESC;

-- 6. Performance score dağılımı
SELECT performance_score, COUNT(*) AS adet
FROM clean_hr
GROUP BY performance_score
ORDER BY performance_score;

-- 7. Joining date yıllara göre dağılım
SELECT EXTRACT(YEAR FROM joining_date) AS yil, COUNT(*) AS adet
FROM clean_hr
WHERE joining_date IS NOT NULL
GROUP BY yil
ORDER BY yil;
```

Transform sonrası clean_hr üzerinde aşağıdaki analiz sorguları çalıştırıldı.

Sonuçlar aşağıdaki gibidir:

Analiz	Sonuç
Toplam kayıt	1000
Boş Name,Age, Salary, Email, Phone, Joining Date	0, 335, 167, 390, 376, 0
Yaş dağılımı	Min: 25 | Max: 50 | Ort: 37.3
Maaş dağılımı	Min: 50000 | Max: 70000 | Ort: 60216.09
Departmana göre çalışan sayısı	Finance: 218 | Sales: 202 | HR: 200 | IT: 193 | Marketing: 187
Performans skoruna göre dağılım	A: 192 | B: 225 | C: 192 | D: 190 | F: 201
Joining Date ve yıllara göre dağılım	2018: 186 | 2019: 385 | 2020: 429

1.5.Sonuç

Bu projede 1000 satırlık messy_HR_data.csv dosyası üzerinde PostgreSQL ve DBeaver kullanılarak eksiksiz bir ETL süreci uygulandı.

Extract aşamasında ham veri, tüm sütunları TEXT tipinde tanımlanmış staging_hr tablosuna aktarıldı. Bu sayede veri hiçbir dönüşüme uğramadan saklandı ve ön analiz sorguları ile kalite sorunları tespit edildi: Age ve Salary sütunlarında metin olarak girilmiş sayısal değerler, Joining Date sütununda 5 farklı tarih formatı ve 390'dan fazla kayıtta geçersiz e-posta verisi.

Transform aşamasında tespit edilen sorunlar SQL CASE ifadeleri ve regex desenleriyle çözüldü. Metin olarak girilmiş sayılar (thirty → 30, SIXTY THOUSAND → 60000) dönüştürüldü, anlamsız değerler (nan, NAN) NULL'a çevrildi, 5 farklı tarih formatı TO_DATE() fonksiyonu ile standart DATE tipine getirildi ve tüm metin alanlarında TRIM() uygulandı.

Load aşamasında dönüştürülen 1000 satır, doğru veri tipleriyle tanımlanmış clean_hr tablosuna tek bir INSERT INTO ... SELECT sorgusuyla yüklendi.

Son olarak veri kalitesi raporu sorguları ile yüklenen verinin doğruluğu doğrulandı; yaş, maaş ve departman bazlı dağılımlar analiz edildi.

2. PROJE 2: VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

2.1.Giriş

Bu proje kapsamında PostgreSQL ve DBeaver kullanılarak e-ticaret veritabanı (1M sipariş, 1000 müşteri, 500 ürün) üzerinde performans analizi ve optimizasyon teknikleri uygulandı. EXPLAIN ANALYZE ile sorgu süreleri ölçüldü, index yönetimi yapıldı ve farklı kullanıcı rolleri için erişim kontrolü kuruldu.

2.2.Index Öncesi Performans Ölçümü

```sql
-- Index OLMADAN sorgu performansı ölçümü

-- 1. Tarihe göre sipariş sorgulama
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE order_date BETWEEN '2020-01-01' AND '2020-12-31';

-- 2. Müşteriye göre sipariş sorgulama
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 500;

-- 3. Ürün kategorisine göre satış toplamı
EXPLAIN ANALYZE
SELECT p.category, SUM(o.total_price) AS toplam_satis
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;

-- 4. Duruma göre sipariş sayısı
EXPLAIN ANALYZE
SELECT status, COUNT(*) FROM orders
GROUP BY status;
```

4 sorgu EXPLAIN ANALYZE ile çalıştırıldı. Tüm sorgularda Seq Scan (tam tablo taraması) gözlemlendi.

Sorguların hızları aşağıda verilmiştir.

	Sorgu 1	Sorgu 2	Sorgu 3	Sorgu 4
Execution Time	82.332 ms	16.234 ms	86.576 ms	42.530 ms

2.3.Index Oluşturma

```sql
CREATE INDEX idx_orders_order_date ON orders(order_date);
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_orders_product_id ON orders(product_id);
CREATE INDEX idx_orders_status ON orders(status);
```

orders tablosuna 4 index eklendi. Index oluşturma işlemi 1 milyon satır için 1.36 saniye sürdü.

2.4.Index Sonrası Performans Ölçümü

Eklenen 4 index sonrasında 4 sorgu tekrardan çalıştırıldı. Sonuçlar aşağıda verilmiştir.

	Sorgu 1	Sorgu 2	Sorgu 3	Sorgu 4
Önce	82.332 ms	16.234 ms	86.576 ms	42.530 ms
Sonra	48.8 ms	0.9 ms	93.7 ms	38.6 ms
Fark	↓%40	↓%94	↑%8	↓%9

Index sonrası sorgu planlarında Index Scan kullanımı gözlemlendi. Özellikle customer_id sorgusunda (Sorgu 2) dramatik iyileşme sağlandı. Sorgu 3'te index sonrası hafif yavaşlama gözlemlendi. PostgreSQL, GROUP BY + JOIN içeren agregasyon sorgularında index yerine Seq Scan'i tercih edebilir.

2.5.Gereksiz Index Tespiti ve Kaldırma

```sql
-- Tüm indexlerin kullanım istatistiklerini görüntüle
SELECT
    schemaname,
    relname AS tablename,
    indexrelname AS indexname,
    idx_scan AS kac_kez_kullanildi,
    idx_tup_read AS okunan_satir,
    idx_tup_fetch AS getirilen_satir
FROM pg_stat_user_indexes
WHERE relname = 'orders'
ORDER BY idx_scan ASC;

-- Index boyutlarını görüntüle
SELECT
    indexrelname AS indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_boyutu
FROM pg_stat_user_indexes
WHERE relname = 'orders';


-- Gereksiz index'i kaldır
DROP INDEX idx_orders_status;

-- Kaldırma sonrası doğrula
SELECT indexrelname AS indexname
FROM pg_stat_user_indexes
WHERE relname = 'orders';
```

pg_stat_user_indexes ile index kullanım istatistikleri incelendi. idx_orders_status en az fayda sağlayan index olarak tespit edildi ve kaldırıldı.

Index	Kullanım	Boyut	Karar
idx_orders_order_date	1	7.2 MB	Tutuldu
idx_orders_customer_id	1	7.1 MB	Tutuldu
idx_orders_product_id	2	7.0 MB	Tutuldu
idx_orders_status	1	6.8 MB	Kaldırıldı

2.6.Erişim Yönetimi ve Roller

```sql
-- Rolleri oluştur
CREATE ROLE readonly_user LOGIN PASSWORD 'readonly123';
CREATE ROLE analyst_user LOGIN PASSWORD 'analyst123';
CREATE ROLE manager_user LOGIN PASSWORD 'manager123';

-- readonly_user: sadece okuma yetkisi
GRANT CONNECT ON DATABASE performans_proje TO readonly_user;
GRANT USAGE ON SCHEMA public TO readonly_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_user;

-- analyst_user: okuma + sorgulama yetkisi
GRANT CONNECT ON DATABASE performans_proje TO analyst_user;
GRANT USAGE ON SCHEMA public TO analyst_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst_user;
GRANT SELECT ON orders, customers, products TO analyst_user;

-- manager_user: tam yetki
GRANT CONNECT ON DATABASE performans_proje TO manager_user;
GRANT USAGE ON SCHEMA public TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO manager_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO manager_user;

-- Rolleri doğrula
SELECT rolname, rolcanlogin, rolsuper
FROM pg_roles
WHERE rolname IN ('readonly_user', 'analyst_user', 'manager_user');


-- Yetkileri doğrula
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee IN ('readonly_user', 'analyst_user', 'manager_user')
ORDER BY grantee, table_name;
```

3 farklı kullanıcı rolü oluşturuldu ve yetkileri atandı.

Rol	Yetki
readonly_user	SELECT
analyst_user	SELECT
manager_user	SELECT, INSERT, UPDATE, DELETE

Rol	Giriş Yapabilir	Süper Kullanıcı
readonly_user	True	False
analyst_user	True	False
manager_user	True	False

2.7.Sonuç

1 milyon satırlık e-ticaret veritabanı üzerinde tam bir performans optimizasyonu süreci uygulandı. EXPLAIN ANALYZE ile index öncesi/sonrası sorgular karşılaştırıldı; customer_id filtreleme sorgusunda %94 hız kazanımı sağlandı. pg_stat_user_indexes ile gereksiz index (idx_orders_status) tespit edilerek kaldırıldı. Son olarak readonly, analyst ve manager rolleri oluşturularak erişim kontrolü kuruldu.